Lecture 3: Reinforcement Learning Motivation & Math

RLHFBOOK.COM

Nathan Lambert

Post-training course.
Chapter 6, Part 1

Lecture 3: Reinforcement learning (mostly the math)

Overview

1. Introduction
2. Key related works
3. Training overview

Core training pipeline

4. Instruction tuning
5. Reward models
6. **Reinforcement learning**
7. Reasoning
8. Direct alignment
9. Rejection sampling

Data & preferences

10. What are preferences
11. Preference data
12. Synthetic data & CAI

Practical considerations

13. Tool use
14. Over-optimization
15. Regularization
16. Evaluation
17. Product & character

Appendices

- A. Definitions
- B. Style & information
- C. Practical issues

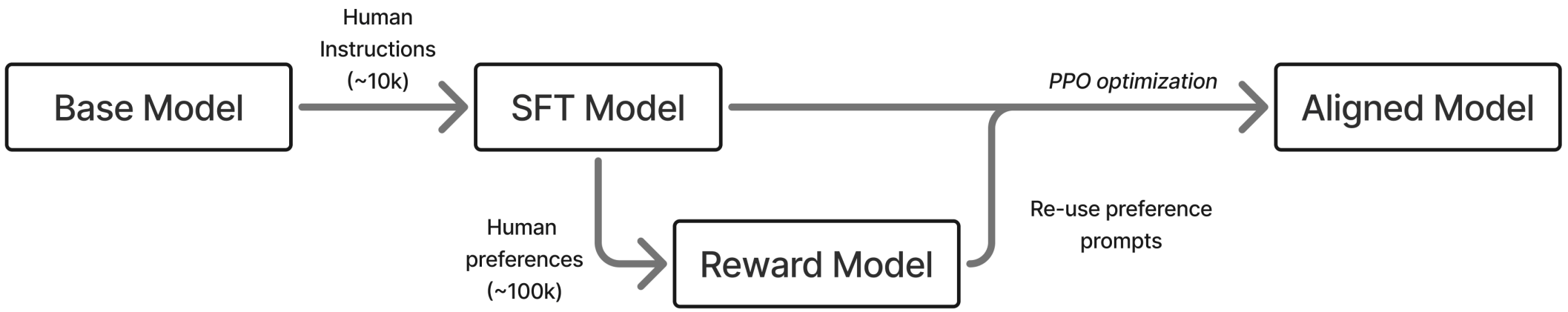
Course home

- rlhfbook.com
- GitHub repo

Where we are in the pipeline

After instruction tuning and training a reward model, the RL step updates the policy against a learned reward signal.

System diagram view.

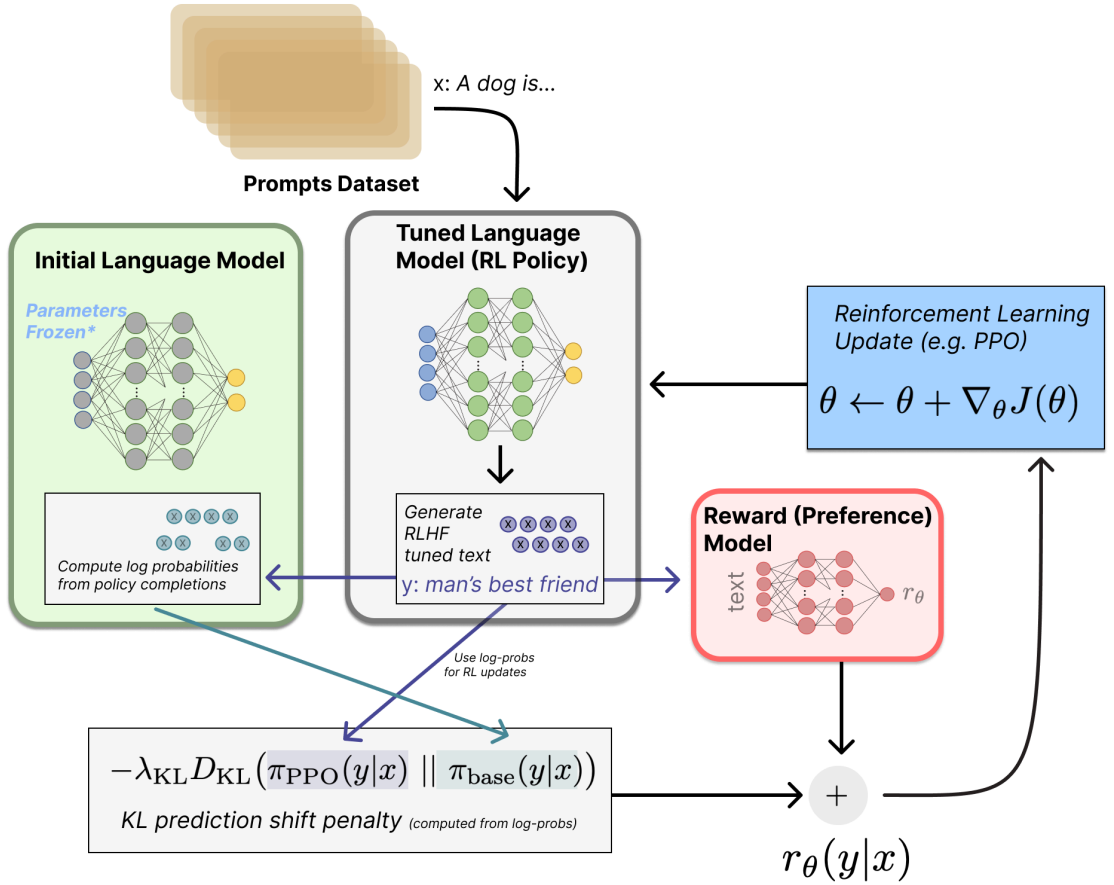


The RLHF training pipeline — the RL step optimizes the policy using the reward model signal.

Where we are in the pipeline

After instruction tuning and training a reward model, the RL step updates the policy against a learned reward signal.

RLHF optimization view.



The RLHF training pipeline — the RL step optimizes the policy using the reward model signal.

What this lecture covers

This lecture covers the **math and theory** of RL for language models. The next lecture covers implementation.

Lecture 3 outline

1. **Motivation** — why RL training matters
2. **RL foundations review** — MDP, return, value functions, policy gradient objective
3. **Policy gradient derivation** — log-derivative trick, step by step
4. **REINFORCE & RLOO** — the simplest policy gradient algorithms
5. **PPO** — trust regions, clipping, GAE, value functions
6. **GRPO & modern variants** — GSPO, CISPO, and the trend toward simplicity
7. **Comparison** — strengths, weaknesses, questions

An under-documented benefit of RL

In lecture 2, we covered rejection sampling and later we'll cover direct alignment algorithms (like DPO). They're simpler, but RL has hard to measure benefits.

Implementing RL is far more complex infrastructure, but the gradient updates it provides "generally help the model a lot." This is hard to quantify, but:

- RL can "fix" rough edges on a model — making outputs more robust, better formatted, and more compatible with serving frameworks like vLLM
- RL operates on a narrow prompt distribution and tends not to "squash" general capabilities — it can be applied surgically to improve specific behaviors without degrading others

Overall, RL losses on language models are robust, scalable, effective, and flexible, which opened large new fields of experimentation. The original method that started us down this path was RLHF work.

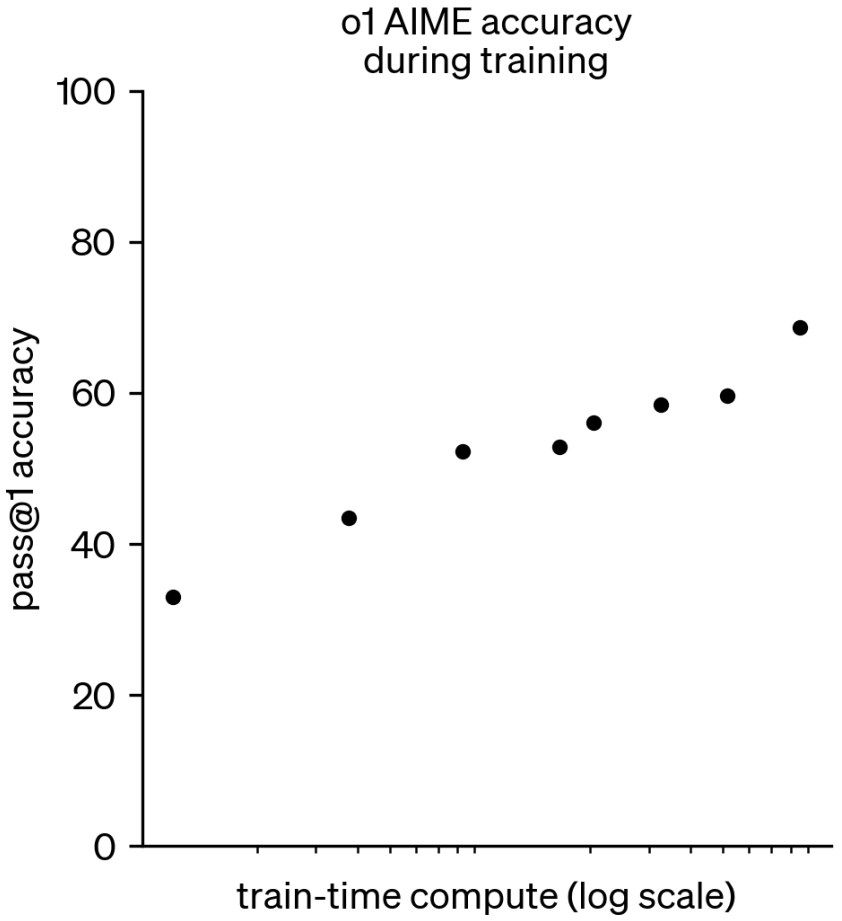
Why RL matters

The scaling RL era

RL is now the **load-bearing step** in training the most capable models.

Reasoning models (o1, DeepSeek R1, etc.) are trained with these exact algorithms on verifiable rewards.

- Train-time compute scaling via RL
- Test-time compute scaling via extended reasoning
- RL went from “cherry on top” to a core capability driver



OpenAI o1: scaling RL training compute improves reasoning performance.

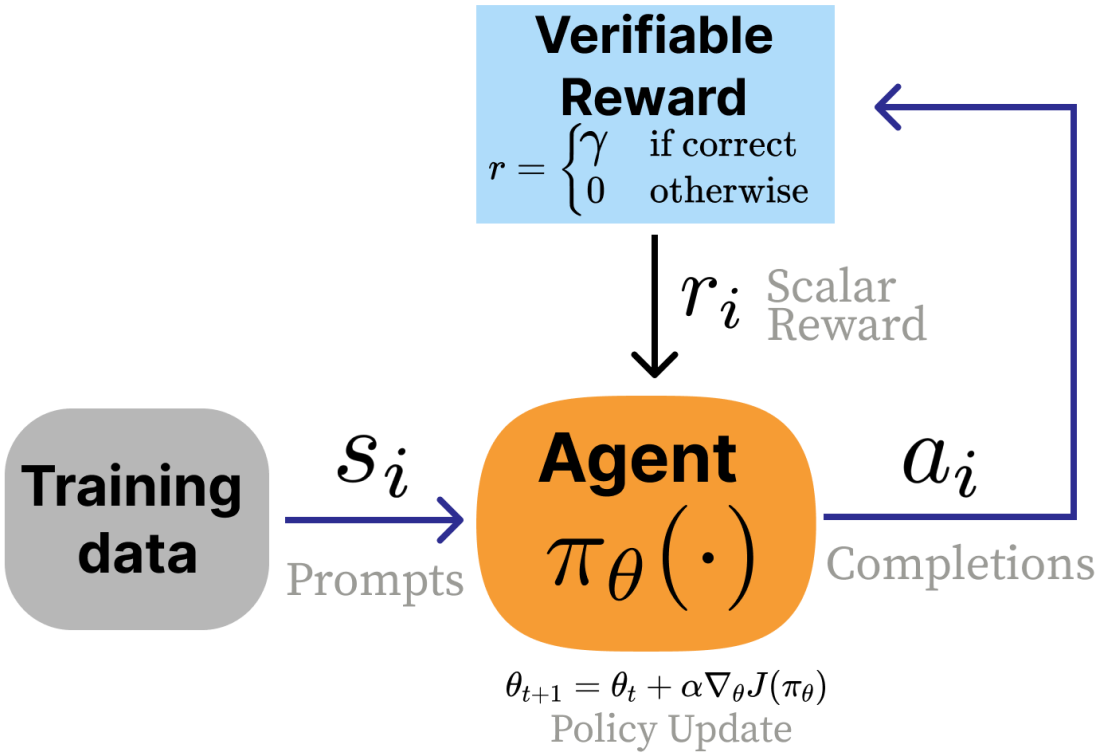
OpenAI, 2024

RLVR: Same algorithms, verifiable rewards

The RL methods in this lecture power **both RLHF and RLVR**:

- **RLHF**: reward model scores subjective quality
- **RLVR**: verification function checks correctness (math, code)

Same policy gradient algorithms, different reward source. We will cover tricks for reasoning training and a bunch of interesting models in depth in a future lecture.



RLVR uses a verification function instead of a reward model, but the RL algorithms are the same.

Lambert et al., 2024

Policy gradients: Core intuitions

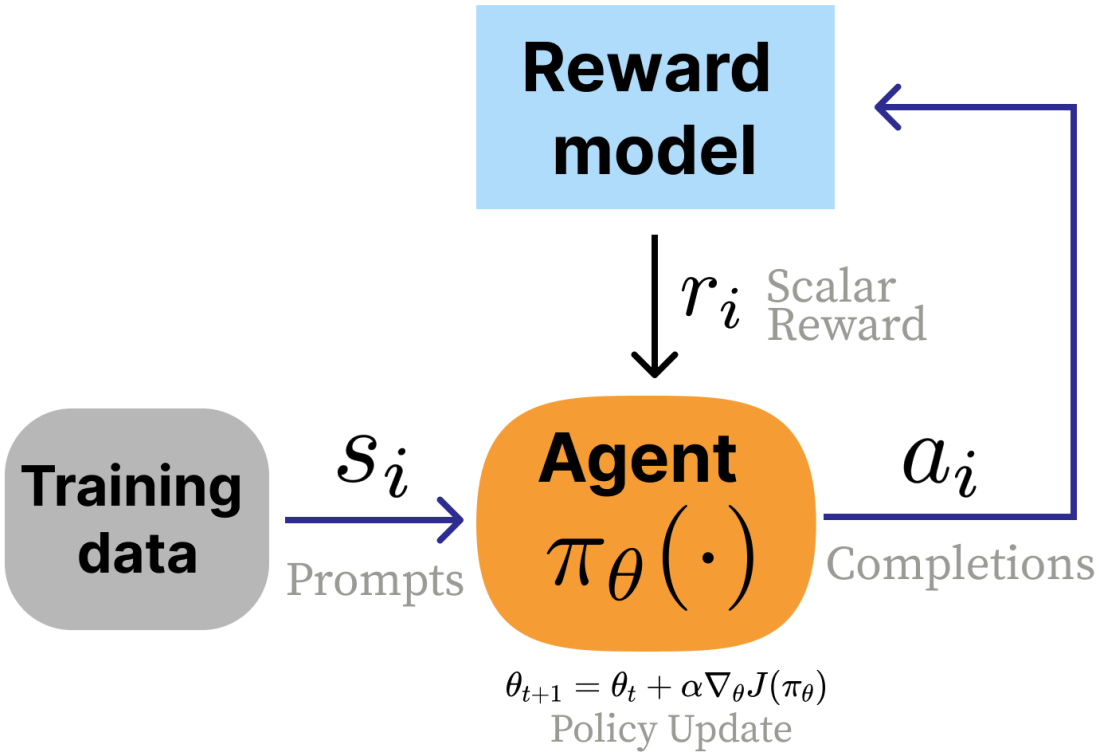
Recall: Classical RL vs. RLHF

Classical RL: agent in an MDP $(\mathcal{S}, \mathcal{A}, P, r, \gamma)$

- Reward is a known function from the environment per step
- Optimize cumulative return: $J(\pi) = \mathbb{E}_{\tau \sim p_\pi}[\sum_t \gamma^t r(s_t, a_t)]$

RLHF: prompts from a dataset, no environment

- Reward is **learned** from human preferences
- **Response-level** reward, regularized with KL penalty
- $J(\pi) = \mathbb{E}\left[r_\theta(x, y)\right] - \beta\, D_{\mathrm{KL}}(\pi \| \pi_{\mathrm{ref}})$



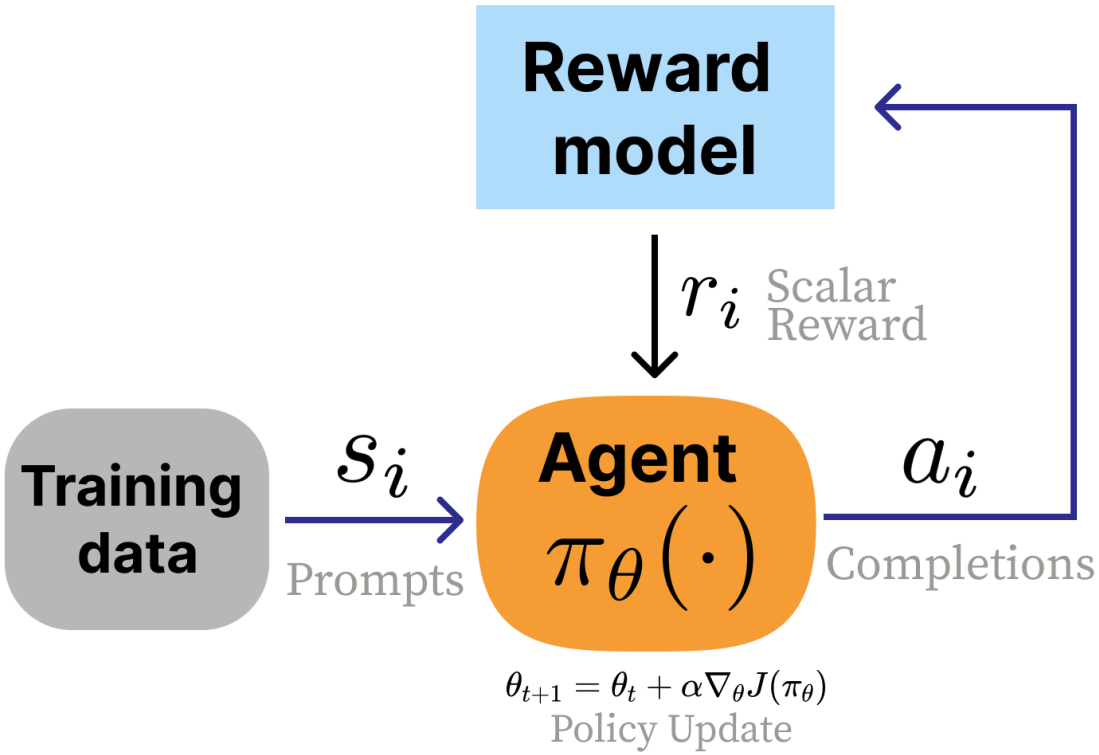
Christiano et al., 2017; Ouyang et al., 2022

Notation in this lecture

This lecture uses (s, a) notation from the reinforcement learning literature, where s denotes states and a denotes actions. In the language model context, you will often see (x, y) instead, where x is the prompt and y is the completion.

The (s, a) framing is more general — these algorithms were designed for sequential decision problems where actions are taken at each timestep. However, many RLHF implementations treat the entire completion as a single action, making the (x, y) notation equally valid.

Notation: r_t for per-step rewards, $R(\tau)$ for trajectory returns, ρ_t for importance-sampling ratios. We reserve ρ for ratios to avoid confusion with rewards.



Language models as episodic MDPs

MDP concept	Language model
State s_t	Prompt + tokens generated so far: $(x, y_{<t})$
Action a_t	Next token y_t
Transition	Deterministic: append token to sequence
Policy $\pi_\theta(a_t \mid s_t)$	LM next-token distribution
Episode	One prompt → completion
Terminal reward	RM score or verifier output
γ	Typically 1.0 (no discounting)

Two views of this MDP: **token-level** (each token is a separate action, used in PPO with GAE) vs. **sequence-level** (entire completion is one action, used in REINFORCE/GRPO).

What are we optimizing?

Choose policy parameters θ that maximize reward **on average** under the current policy.

Let $p_\theta(\tau)$ be the trajectory distribution induced by the current policy.

$$J(\theta) = \mathbb{E}_{\tau \sim p_\theta}[R(\tau)]$$

In practice, we estimate this expectation with sampled rollouts:

$$J(\theta) \approx \frac{1}{N} \sum_{i=1}^{N} R(\tau_i), \qquad \tau_i \sim p_\theta$$

We craft a gradient/derivative that lets us optimize this.

How a policy gradient works

Make actions more likely when they lead to better outcomes.

$$\Delta\theta \propto \Psi_t \, \nabla_\theta \log \pi_\theta(a_t \mid s_t)$$

Read it as two questions answered at once:

- $\nabla_\theta \log \pi_\theta(a_t \mid s_t)$ — **which action?**
- Ψ_t — **how good was it?** A scalar scoring the outcome.

How a policy gradient works

Make actions more likely when they lead to better outcomes.

$$\Delta\theta \propto \Psi_t \, \nabla_\theta \log \pi_\theta(a_t \mid s_t)$$

Read it as two questions answered at once:

- $\nabla_\theta \log \pi_\theta(a_t \mid s_t)$ — **which action?** How each parameter influenced the probability of taking action a_t in state s_t. This connects the outcome back to the knobs that caused it.
- Ψ_t — **how good was it?** A scalar scoring the outcome. Positive means good, negative means bad, magnitude says how much.

> *An oversimplification (e.g. intuition in case of batch size of 1): the gradient is a vector with one entry per parameter. A positive entry means "increasing this parameter made the action more likely," a negative entry means the opposite. In practice, the update averages over many such vectors — what survives is the net vote across the batch.*

Multiply them: $\Psi_t > 0$ updates parameters to make a_t more likely, $\Psi_t < 0$ updates them to make it less likely.

The rest of this section is about choosing a smart Ψ_t — different choices (total return, advantage, TD residual) trade off variance and bias, but all plug into this same update.

The policy gradient equation

A preview of where we end up:

$$\nabla_\theta J(\theta) = \mathbb{E}_{\tau \sim p_\theta} \left[\sum_{t=0}^{T} \Psi_t \nabla_\theta \log \pi_\theta(a_t \mid s_t) \right]$$

The core idea is that we *sample over trials in the environment* and **estimate** the gradient.

Where Ψ_t is the **learning signal** telling the optimizer how good the action was. The choice of Ψ_t determines the algorithm's variance, bias, and compute cost.

Key RL quantities

Three quantities appear throughout this lecture:

Value function $V(s)$: expected future return from state s

$$V(s) = \mathbb{E}\big[G_t \mid S_t = s\big] \quad \text{where } G_t = \sum_{k=0}^{\infty} \gamma^k r_{t+k}$$

Action-value $Q(s, a)$: expected return after taking action a in state s

$$Q(s, a) = \mathbb{E}[G_t \mid S_t = s, A_t = a]$$

Advantage $A(s, a) = Q(s, a) - V(s)$: how much better is action a compared to average? Positive → reinforce, negative → suppress, zero → no update.

In RLHF: often $\gamma = 1$ (no discounting) because the unit of optimization is the full completion.

What Ψ_t can be

Popular choices for Ψ_t (rewards can also be discounted by γ):

	Ψ_t	Description	Variance	Bias
1.	$R(\tau) = \sum_{t=0}^{T} r_t$	Total trajectory reward	Highest	None
2.	$\sum_{t'=t}^{T} r_{t'}$	Future return from t (the return, G_t)	High	None
3.	$G_t - b(s_t)$	Baselined return	Lower	None
4.	$Q^\pi(s_t, a_t)$	State-action value function	Med	Depends
5.	$A^\pi(s_t, a_t) = Q - V$	Advantage function	Low (with good V)	None
6.	$r_t + \gamma V(s_{t+1}) - V(s_t)$	TD residual	Low	Some

A *baseline* $b(s_t)$ is any value subtracted from the reward signal to reduce variance. We'll show below that this does not change the expected gradient.

Schulman et al., 2015

Deriving the policy gradient

Setup: Differentiating an expectation

Ideally, we want $\nabla_\theta J(\theta)$, but we can't do that (the state sampling distribution itself depends on θ).

Written as an integral, the objective is:

$$J(\theta) = \mathbb{E}_{\tau \sim p_\theta}[R(\tau)] = \int_\tau p_\theta(\tau) R(\tau)\, d\tau$$

Setup: Differentiating an expectation

Ideally, we want $\nabla_\theta J(\theta)$, but we can't do that (the state sampling distribution itself depends on θ).

Written as an integral, the objective is:

$$J(\theta) = \mathbb{E}_{\tau \sim p_\theta}[R(\tau)] = \int_\tau p_\theta(\tau) R(\tau)\, d\tau$$

Taking the gradient directly:

$$\nabla_\theta J(\theta) = \int_\tau \nabla_\theta p_\theta(\tau) R(\tau)\, d\tau$$

We can sample trajectories from $p_\theta(\tau)$, but not from $\nabla_\theta p_\theta(\tau)$, since it is not a probability distribution.

The derivation is a few key tricks to let us approximate or compute this.

Applying the idea of computing the gradient from trajectories

Substituting the log-derivative identity (more on this soon) into the gradient:

$$\begin{aligned}
\nabla_\theta J(\theta) &= \int_\tau \nabla_\theta p_\theta(\tau) R(\tau)\, d\tau && \text{differentiate the objective} \\
&= \int_\tau p_\theta(\tau) R(\tau) \nabla_\theta \log p_\theta(\tau)\, d\tau && \text{log-derivative identity (chain rule)} \\
&= \mathbb{E}_{\tau \sim p_\theta}[R(\tau) \nabla_\theta \log p_\theta(\tau)] && \text{expectation form}
\end{aligned}$$

Now we can estimate this with Monte Carlo sampling!

Applying the idea of computing the gradient from trajectories

The only non-obvious step is the middle line:

$$\nabla_\theta \log p_\theta(\tau) = \frac{\nabla_\theta p_\theta(\tau)}{p_\theta(\tau)}$$
$$\Rightarrow \quad \nabla_\theta p_\theta(\tau) = p_\theta(\tau) \nabla_\theta \log p_\theta(\tau)$$

This is the **log-derivative identity**, derived from the chain rule for $\log$.

It rewrites the gradient of a probability in terms of a log-probability, which is much easier to work with.

Applying the idea of computing the gradient from trajectories

Where did the $p_\theta(\tau)$ term go?

It became the **sampling distribution** inside the expectation:

$$\int p_\theta(\tau) f(\tau)\, d\tau = \mathbb{E}_{\tau \sim p_\theta}[f(\tau)]$$

Monte Carlo then estimates that expectation by sampling trajectories from the policy:

$$\mathbb{E}_{\tau \sim p_\theta}[f(\tau)] \approx \frac{1}{N} \sum_{i=1}^{N} f(\tau_i), \qquad \tau_i \sim p_\theta$$

So there is no explicit $p_\theta(\tau_i)$ term in code: **more likely trajectories already appear more often in the sampled batch**.

At a high level, rollout generation handles the sampling, and the loss code handles $R(\tau_i)\, \nabla_\theta \log p_\theta(\tau_i)$.

Expanding log probability of trajectory to compute ∇_θ

The trajectory probability factorizes:

$$p_\theta(\tau) = d_0(s_0) \prod_{t=0}^{T} \pi_\theta(a_t \mid s_t)\, p(s_{t+1} \mid s_t, a_t)$$

Taking the log:

$$\log p_\theta(\tau) = \log d_0(s_0) + \sum_{t=0}^{T} \log \pi_\theta(a_t \mid s_t) + \sum_{t=0}^{T} \log p(s_{t+1} \mid s_t, a_t)$$

For language models, this is the familiar autoregressive pattern: a sequence log-probability becomes a sum of token log-probabilities.

Expanding log probability of trajectory to compute ∇_θ

$$\log p_\theta(\tau) = \log d_0(s_0) + \sum_{t=0}^{T} \log \pi_\theta(a_t \mid s_t) + \sum_{t=0}^{T} \log p(s_{t+1} \mid s_t, a_t)$$

Now take the gradient w.r.t. θ:

- $\nabla_\theta \log d_0(s_0) = 0$ — initial state distribution doesn't depend on θ
- $\nabla_\theta \log p(s_{t+1} \mid s_t, a_t) = 0$ — environment dynamics don't depend on θ
- Only $\nabla_\theta \log \pi_\theta(a_t \mid s_t)$ survives!

$$\nabla_\theta \log p_\theta(\tau) = \sum_{t=0}^{T} \nabla_\theta \log \pi_\theta(a_t \mid s_t)$$

Expanding log probability of trajectory to compute ∇_θ

$$\log p_\theta(\tau) = \log d_0(s_0) + \sum_{t=0}^{T} \log \pi_\theta(a_t \mid s_t) + \sum_{t=0}^{T} \log p(s_{t+1} \mid s_t, a_t)$$

Now take the gradient w.r.t. θ:

- $\nabla_\theta \log d_0(s_0) = 0$ — initial state distribution doesn't depend on θ
- $\nabla_\theta \log p(s_{t+1} \mid s_t, a_t) = 0$ — environment dynamics don't depend on θ
- Only $\nabla_\theta \log \pi_\theta(a_t \mid s_t)$ survives!

$$\nabla_\theta \log p_\theta(\tau) = \sum_{t=0}^{T} \nabla_\theta \log \pi_\theta(a_t \mid s_t)$$

In language-model code, this is why you repeatedly see:

```
seq_log_probs = (token_log_probs * completion_mask).sum(dim=-1)
loss = -(seq_log_probs * advantages).mean()
loss.backward()
```

Autodiff turns that summed log-probability into the corresponding sum of per-token gradients!

From full return to future rewards

An action at time t can't affect past rewards. So instead of weighting by the full trajectory reward $R(\tau)$:

$$\nabla_\theta J = \mathbb{E}_{\tau \sim p_\theta}\left[\sum_{t=0}^{T} R(\tau) \nabla_\theta \log \pi_\theta(a_t \mid s_t)\right]$$

We can replace $R(\tau)$ with the **return-to-go** $G_t = \sum_{t'=t}^{T} r_{t'}$ — only future rewards from t onward:

$$\nabla_\theta J = \mathbb{E}_{\tau \sim p_\theta}\left[\sum_{t=0}^{T} G_t \nabla_\theta \log \pi_\theta(a_t \mid s_t)\right]$$

This doesn't change the expected gradient, but removes noise from past rewards that the current action couldn't have influenced. This is the step from Ψ_t option 1 → option 2 in our taxonomy.

Baselines keep the estimator unbiased

Raw returns are noisy. The same action can appear in trajectories with very different total rewards because of later sampled actions and, in classical RL, environment randomness.

The role of a baseline is to **center** that noisy signal: instead of asking “was the return high?”, we ask “was it higher or lower than expected for this state?”

Baselines keep the estimator unbiased

Raw returns are noisy. The same action can appear in trajectories with very different total rewards because of later sampled actions and, in classical RL, environment randomness.

The role of a baseline is to **center** that noisy signal: instead of asking “was the return high?”, we ask “was it higher or lower than expected for this state?”

Use a centered return:

$$\Psi_t = G_t - b(s_t)$$

where the baseline $b(s_t)$ depends only on the state, not on which action was sampled.

Subtracting this baseline doesn’t change the expected gradient:

Baselines keep the estimator unbiased

Raw returns are noisy. The same action can appear in trajectories with very different total rewards because of later sampled actions and, in classical RL, environment randomness.

Use a centered return, $\Psi_t = G_t - b(s_t)$.

Subtracting this baseline doesn't change the expected gradient:

$$\mathbb{E}[(G_t - b(s))\nabla_\theta \log \pi_\theta(a \mid s)] = \mathbb{E}[G_t \nabla_\theta \log \pi_\theta(a \mid s)] - \mathbb{E}[b(s)\nabla_\theta \log \pi_\theta(a \mid s)]$$

The first term is the original estimator. The second vanishes:

$$\begin{aligned}
\mathbb{E}[b(s)\nabla_\theta \log \pi_\theta(a \mid s)] &= b(s) \cdot \sum_a \nabla_\theta \pi_\theta(a \mid s) \\
&= b(s) \cdot \nabla_\theta \underbrace{\sum_a \pi_\theta(a \mid s)}_{=1} \\
&= 0
\end{aligned}$$

The gradient of a normalized distribution sums to zero — so we're free to subtract **any function of state** as a baseline. This is why Ψ_t options 3–5 reduce variance without introducing bias.

Recall: What Ψ_t can be

Popular choices for Ψ_t (rewards can also be discounted by γ):

	Ψ_t	Description	Variance	Bias
1.	$R(\tau) = \sum_{t=0}^{T} r_t$	Total trajectory reward	Highest	None
2.	$\sum_{t'=t}^{T} r_{t'}$	Future return from t (the return, G_t)	High	None
3.	$\sum_{t'=t}^{T} r_{t'} - b(s_t)$	Baselined return	Lower	None
4.	$Q^{\pi}(s_t, a_t)$	State-action value function	Med	Depends
5.	$A^{\pi}(s_t, a_t) = Q - V$	Advantage function	**Lowest**	None
6.	$r_t + \gamma V(s_{t+1}) - V(s_t)$	TD residual	Low	Some

A *baseline* $b(s_t)$ is any value subtracted from the reward signal to reduce variance — we'll show why this is unbiased shortly.

Schulman et al., 2015

The policy gradient estimator

Combining the log-derivative trick, return-to-go, and baseline subtraction:

$$\nabla_\theta J(\theta) = \mathbb{E}_{\tau \sim p_\theta}\left[\sum_{t=0}^{T} \Psi_t \nabla_\theta \log \pi_\theta(a_t \mid s_t)\right]$$

This is the **policy gradient theorem**. Every algorithm in this lecture is an instantiation with a specific choice of Ψ_t and regularization.

For language models, this means the sum runs over generated tokens in the completion, while Ψ_t determines how each token is weighted.

Policy gradient derivation recap

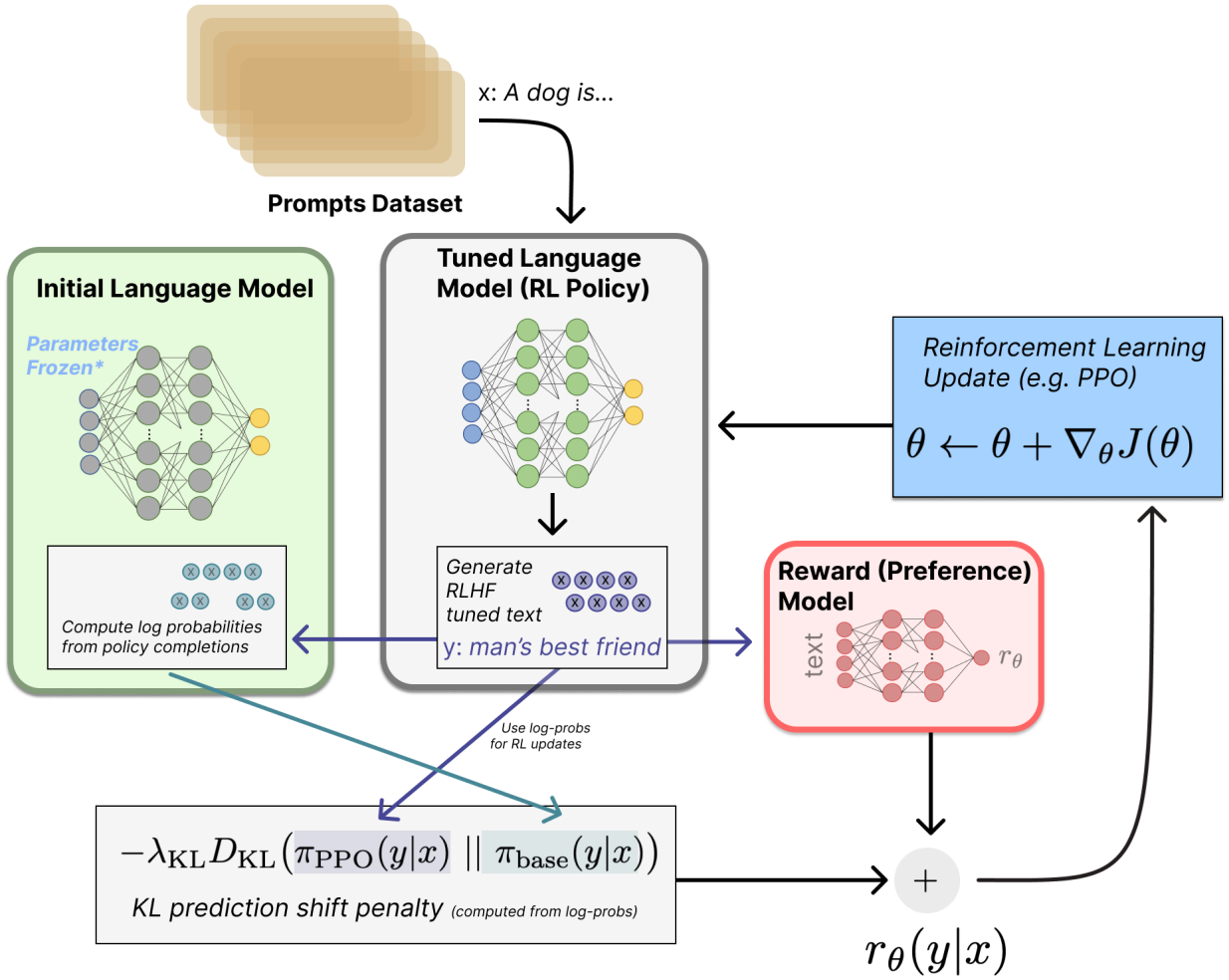
$$
\begin{aligned}
J(\theta) &= \mathbb{E}_{\tau \sim p_\theta}[R(\tau)] && \text{objective} \\
&= \int_\tau p_\theta(\tau) R(\tau)\, d\tau && \text{integral form} \\
\nabla_\theta J(\theta) &= \int_\tau \nabla_\theta p_\theta(\tau) R(\tau)\, d\tau && \text{differentiate} \\
&= \mathbb{E}_{\tau \sim p_\theta}\left[\sum_{t=0}^{T} \Psi_t \nabla_\theta \log \pi_\theta(a_t \mid s_t) \right] && \text{final estimator}
\end{aligned}
$$

- Log-derivative trick turns $\nabla p_\theta(\tau)$ into a sampleable expectation
- Expanding $\log p_\theta(\tau)$ gives a sum of policy log-prob terms
- Ψ_t is the learning signal: return, advantage, or TD-style estimate
- In code: sample rollouts, sum token log-probs, weight by Ψ_t, average

Wiring RLHF for language models

The RLHF training loop

Before diving into specific algorithms, let's revisit the full RLHF setup they plug into.



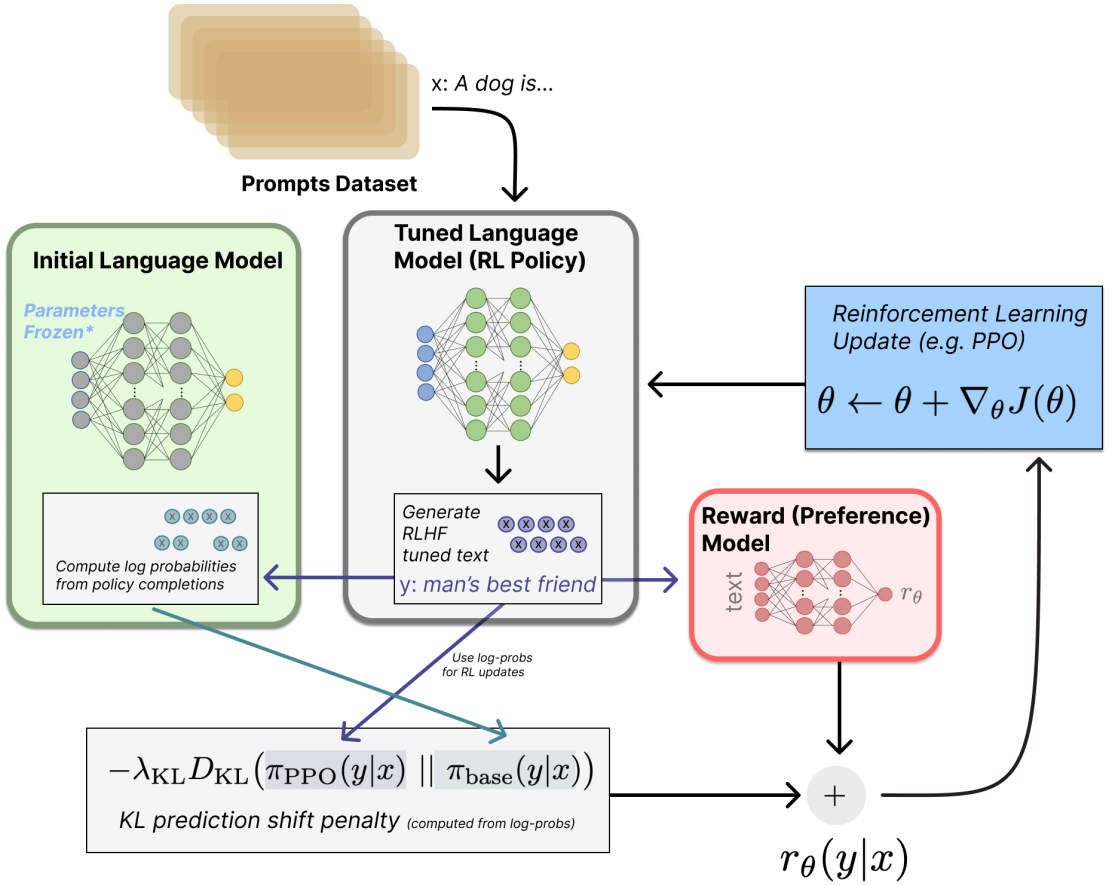
The RLHF training pipeline — the RL step optimizes the policy using the reward model signal.

The reference model

The **reference model** π_{ref} is a frozen copy of the policy at the start of RL training (typically the SFT checkpoint).

It serves one purpose: **anchor the policy so it doesn't drift too far during optimization.**

Without it, the policy can exploit the reward model — finding high-scoring outputs that are degenerate or repetitive (reward hacking).



KL divergence as regularization

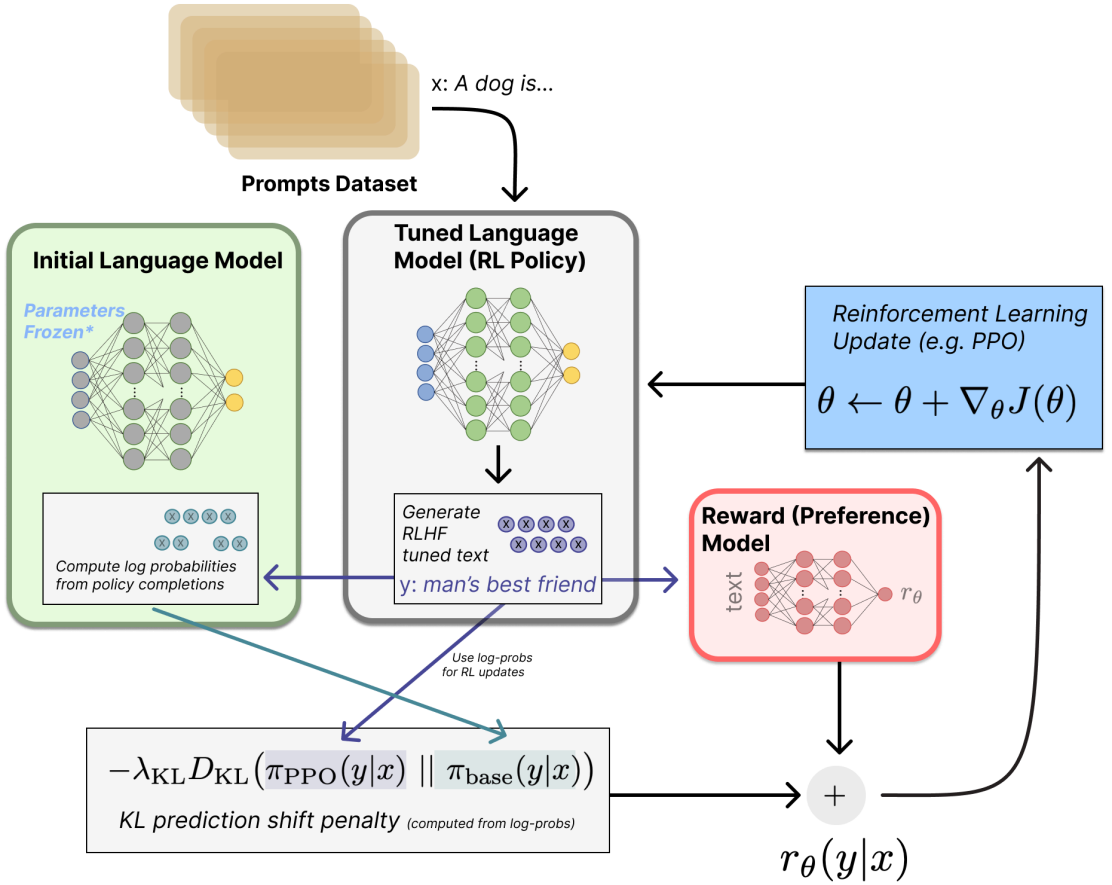
The KL penalty measures how far the current policy has drifted from the reference at each token:

$$\mathrm{KL}_t = \log \pi_\theta(a_t \mid s_t) - \log \pi_{\mathrm{ref}}(a_t \mid s_t)$$

In practice:

1. **Generate** a completion from π_θ
2. **Compute log-probs** of those same tokens under both π_θ and π_{ref}
3. **Subtract** to get the per-token KL

The shaped reward becomes: $\tilde{r}_t = -\beta\,\mathrm{KL}_t$ for intermediate tokens, $\tilde{r}_T = R(\tau) - \beta\,\mathrm{KL}_T$ for the final token.

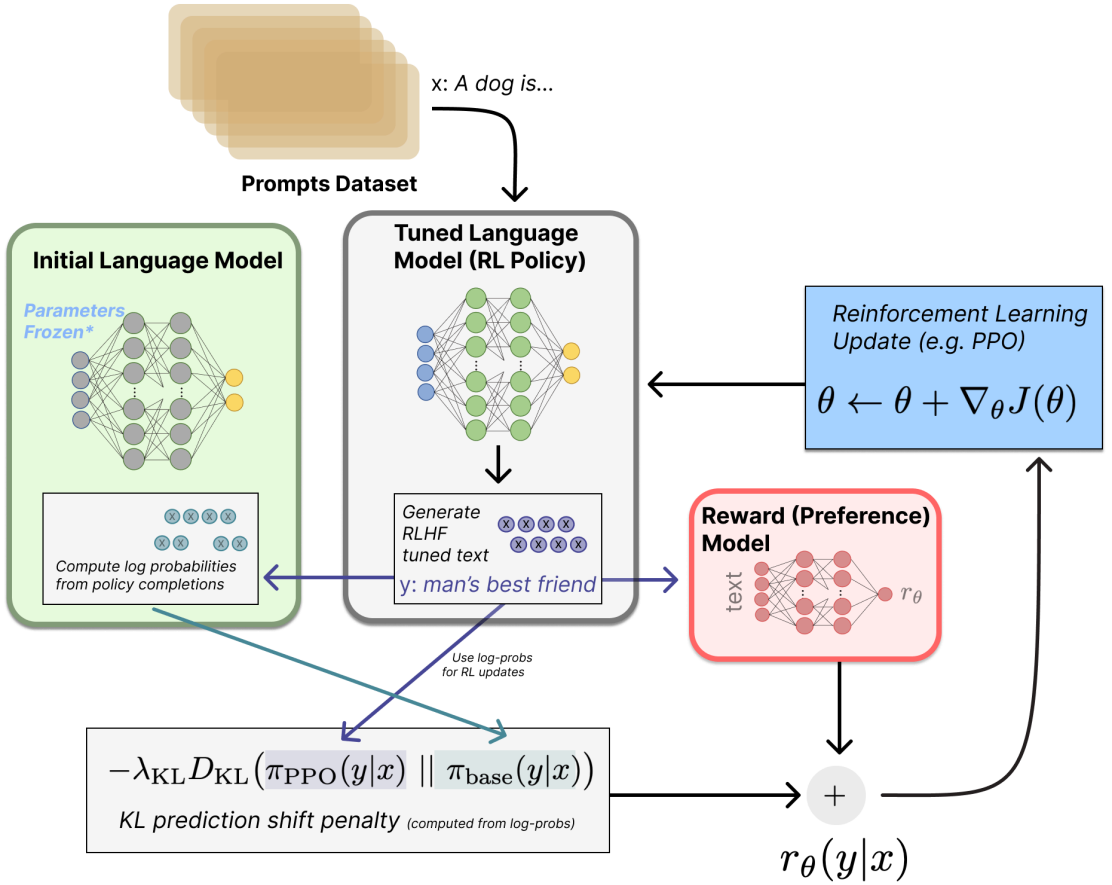


KL in RLHF vs. RLVR

RLHF (reward model signal): KL penalty is critical — the reward model is a learned proxy, and the policy will exploit any imperfections without regularization.

RLVR (verifiable rewards, e.g. math correctness): KL is often reduced or removed entirely. The reward is ground truth, so there's less to exploit. Some RLVR setups (e.g. DeepSeek R1) drop the reference model altogether, saving memory.

The trend: as reward signals become more reliable, the need for KL regularization decreases.



Two “old” policies — don’t confuse them

	$\pi_{\theta_{\text{old}}}$	π_{ref}
What	Policy at last rollout	Policy at start of RL training (SFT checkpoint)
Updates	Every batch (or every K steps)	Never (frozen)
Used for	Importance-sampling ratio ρ_t	KL penalty
If dropped	Must use 1 gradient step per batch (on-policy)	Risk of reward hacking

In some implementations, “old logprobs” refers to the generation-time logprobs — the model has simply been updated since those logprobs were computed, not a separate model copy.

REINFORCE

REINFORCE

REINFORCE is the simplest instantiation of the policy gradient.

It is the **Monte Carlo form of policy gradient**: sample trajectories, compute their returns, and use those sampled returns to weight the log-prob gradients.

Williams, 1992

REINFORCE

REINFORCE is the simplest instantiation of the policy gradient.

It is the **Monte Carlo form of policy gradient**: sample trajectories, compute their returns, and use those sampled returns to weight the log-prob gradients.

> *The name is an acronym for "REward Increment = Nonnegative Factor X Offset Reinforcement X Characteristic Eligibility."*

Three components:

1. **Nonnegative factor** (α): the learning rate
2. **Offset reinforcement** $(G_t - b(s_t))$: centered return for stability
3. **Characteristic eligibility** $(\nabla \log \pi)$: how to attribute learning per token

Williams, 1992

REINFORCE

REINFORCE is the simplest instantiation of the policy gradient.

It is the **Monte Carlo form of policy gradient**: sample trajectories, compute their returns, and use those sampled returns to weight the log-prob gradients.

> *The name is an acronym for "REward Increment = Nonnegative Factor X Offset Reinforcement X Characteristic Eligibility."*

Three components:

1. **Nonnegative factor** (α): the learning rate
2. **Offset reinforcement** $(G_t - b(s_t))$: centered return for stability
3. **Characteristic eligibility** $(\nabla \log \pi)$: how to attribute learning per token

The update rule:

$$\Delta\theta = \alpha \left(G_t - b(s_t)\right) \nabla_\theta \log \pi_\theta(a_t \mid s_t)$$

Williams, 1992

The baseline problem

Without a baseline, the gradient weights each action by its raw return G_t. This doesn't tell you whether an action was **better or worse than expected** — just how good the overall outcome was.

The gradient is high variance because the raw return mixes the quality of the action with the quality of the state. A good action in a bad state and a bad action in a good state can produce similar returns.

The baseline solution

Subtracting a baseline $b(s)$ centers the signal: now the gradient weight is $(G_t - b)$, which answers "was this action better or worse than expected from this state?"

$$\mathbb{E}_{a\sim\pi(\cdot|s)}[b(s)\nabla_\theta \log \pi_\theta(a \mid s)] = b(s) \cdot \mathbb{E}_{a\sim\pi(\cdot|s)}[\nabla_\theta \log \pi_\theta(a \mid s)] = 0$$

Because $b(s)$ does not depend on which action was sampled, it factors out and cancels. The expected gradient is unchanged — but the variance drops dramatically.

REINFORCE with baseline

The full REINFORCE gradient:

$$\nabla_\theta J(\theta) = \mathbb{E}_{\tau \sim p_\theta}\left[\sum_{t=0}^{T}(G_t - b(s_t))\nabla_\theta \log \pi_\theta(a_t \mid s_t)\right]$$

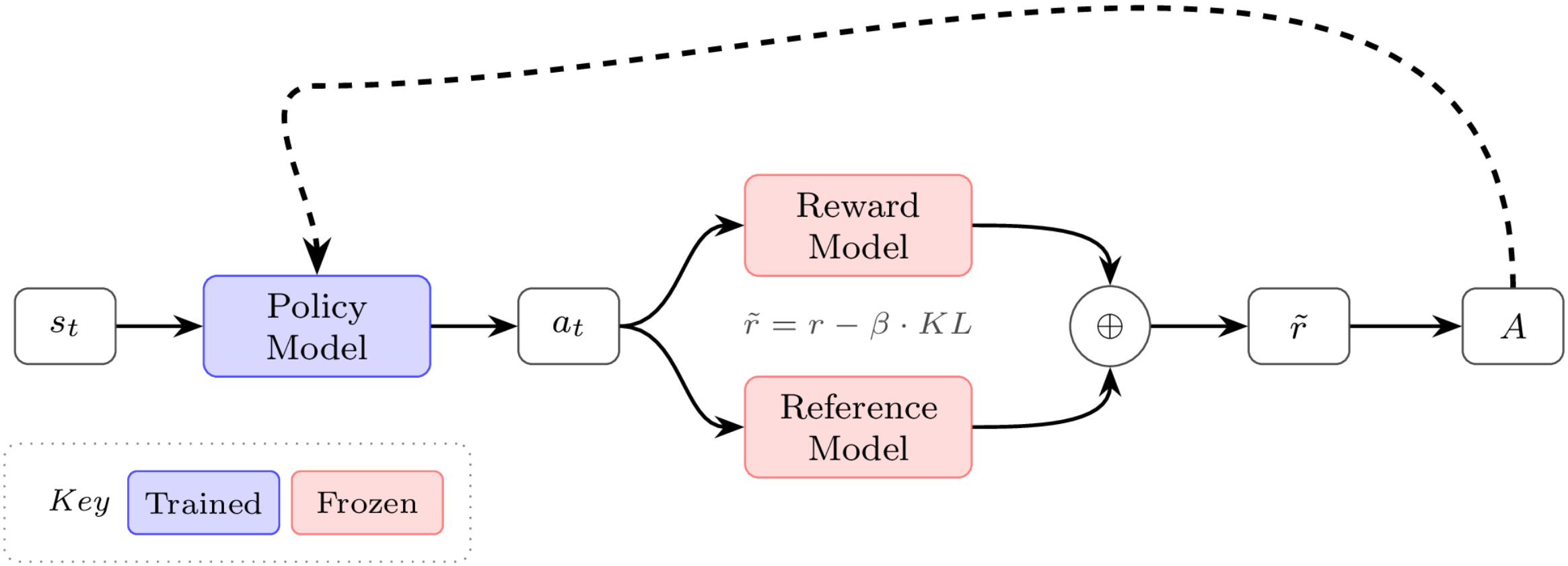
Here, $G_t - b(s_t)$ is already an advantage estimate: how much better the realized return was than expected from state s_t.

Common baselines:

- **Average reward** over the batch (simplest)
- **Moving average** of recent rewards
- **Learned value function** $V_\phi(s)$ (optional — bridges to actor-critic methods)

Basic REINFORCE needs no critic — just Monte Carlo returns and a simple baseline. Adding a learned V_ϕ can reduce variance further but introduces the complexity of training a second model (... and moves towards PPO).

REINFORCE architecture



Basic REINFORCE architecture for language models. The shaped reward combines the reward model score with a KL penalty from the reference model.

REINFORCE leave-one-out (RLOO)

RLOO: Leave-one-out baseline

Key idea: generate K completions per prompt. Use the other $K - 1$ rewards as the baseline:

$$b(s, a_k) = \frac{1}{K-1} \sum_{i=1, i \neq k}^{K} R(s, a_i)$$

The advantage for completion k:

$$\hat{A}(s, a_k) = R(s, a_k) - b(s, a_k)$$

This is a **per-prompt** baseline that naturally captures prompt difficulty — hard prompts get low rewards across all completions, so the baseline is low.

Detail: Like REINFORCE without a critic, this same sequence-level advantage is broadcast to every token in the completion.

Ahmadian et al., 2024

RLOO worked example

$K = 4$ completions for one prompt, with rewards $[0.8, 0.3, 0.6, 0.5]$:

Completion	Reward	Baseline (avg of others)	Advantage
1	0.8	$(0.3 + 0.6 + 0.5)/3 = 0.467$	$+0.333$
2	0.3	$(0.8 + 0.6 + 0.5)/3 = 0.633$	-0.333
3	0.6	$(0.8 + 0.3 + 0.5)/3 = 0.533$	$+0.067$
4	0.5	$(0.8 + 0.3 + 0.6)/3 = 0.567$	-0.067

Completion 1 (best) gets reinforced. Completion 2 (worst) gets suppressed. Completions 3 and 4 get small updates.

REINFORCE / RLOO summary

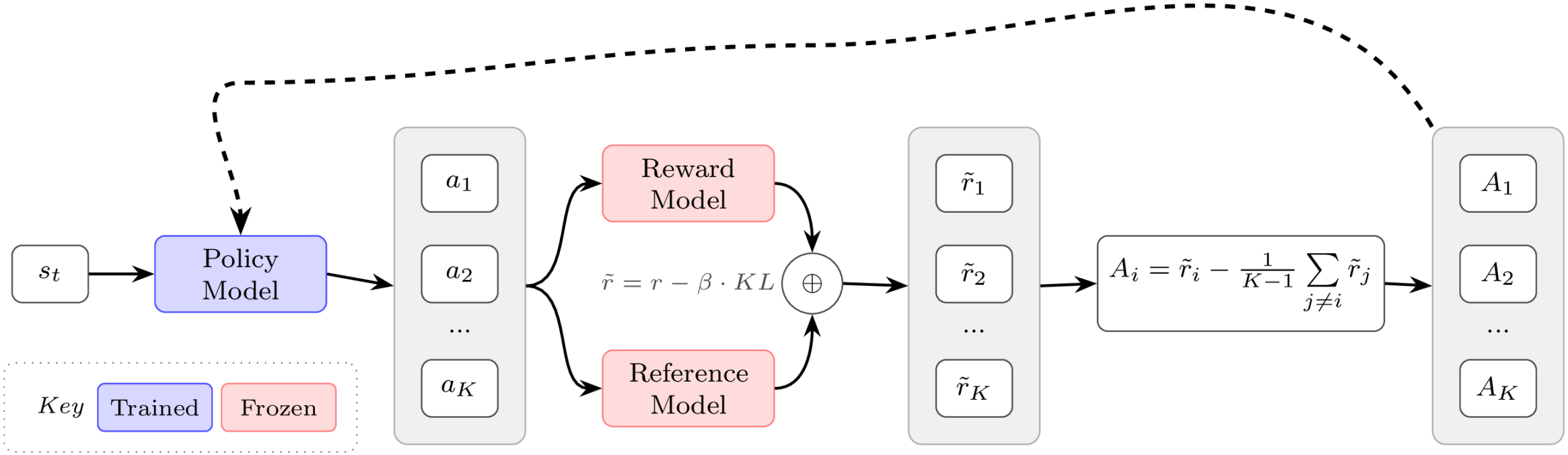
REINFORCE: Simplest policy gradient. Needs a baseline to reduce variance. No value function required.

RLOO: REINFORCE + a smart, per-prompt leave-one-out baseline. Multiple completions per prompt provide the baseline for free.

Both are the foundation for everything that follows:

- Linked intuitively to GRPO and other grouped-completion algorithms
- No value function needed (saves memory/compute)
- Simple to implement and debug
- Strong baselines in practice (Ahmadian et al., 2024) (Wang et al., 2024)

RLOO architecture



RLOO architecture for language models. Same structure as REINFORCE, but multiple completions per prompt provide the leave-one-out baseline.

Proximal policy optimization (PPO)

If REINFORCE is so simple, why PPO?

All the algorithms in this lecture are **on-policy**: they generate fresh rollouts from the current policy each batch, then update on those rollouts. This is in contrast to off-policy RL methods (e.g. DQN) that store and replay old experience.

If REINFORCE is so simple, why PPO?

All the algorithms in this lecture are **on-policy**: they generate fresh rollouts from the current policy each batch, then update on those rollouts. This is in contrast to off-policy RL methods (e.g. DQN) that store and replay old experience.

The problem: vanilla policy gradient is sensitive to step size. Too large an update and the policy can collapse; too small and training is painfully slow. TRPO (Schulman et al., 2015) solved this with a hard trust-region constraint, but required expensive second-order optimization.

If REINFORCE is so simple, why PPO?

All the algorithms in this lecture are **on-policy**: they generate fresh rollouts from the current policy each batch, then update on those rollouts. This is in contrast to off-policy RL methods (e.g. DQN) that store and replay old experience.

The problem: vanilla policy gradient is sensitive to step size. Too large an update and the policy can collapse; too small and training is painfully slow. TRPO (Schulman et al., 2015) solved this with a hard trust-region constraint, but required expensive second-order optimization.

Proximal Policy Optimization **(PPO)** (Schulman et al., 2017) gets TRPO-like stability with a simple clipped objective — and because the clipping keeps updates conservative, you can safely take multiple gradient steps per batch of rollouts, improving sample efficiency.

What can we do with more conservative gradients?

Extract more signal from the batch! This introduces new problems:

- How do we constrain the updates over multiple gradient updates?
- How do we take the policy gradient if the data has drifted off-policy?

PPO core idea 1: Constrained updates

Large gradient steps can destroy the policy (instability, over-optimization, etc.)

The solution: **trust regions** — limit how far the policy can move in a single update.

Schulman et al., 2017

PPO core idea 2: Importance sampling

We want to take multiple gradient steps on a batch, but the data came from an old policy $\pi_{\theta_{\text{old}}}$.

Define the **importance sampling ratio**:

$$\rho_t(\theta) = \frac{\pi_\theta(a_t \mid s_t)}{\pi_{\theta_{\text{old}}}(a_t \mid s_t)}$$

- If $\rho_t = 1$: current and old policy agree on this action
- If $\rho_t > 1$: current policy assigns higher probability than old
- If $\rho_t < 1$: current policy assigns lower probability than old

This ratio reweights old-policy samples to estimate new-policy gradients. Plugging that ratio into the earlier policy-gradient form gives a surrogate objective we can optimize on old-policy data.

The surrogate objective

Using importance sampling, the policy gradient becomes:

$$J(\theta) = \mathbb{E}_t\left[\rho_t(\theta)\hat{A}_t\right]$$

Intermediate problem: without constraints, maximizing this can take arbitrarily large steps — the ratio ρ_t can diverge far from 1, making the estimate unreliable.

The PPO clipped objective (put pieces together)

PPO clips the ratio to prevent large updates — a practical surrogate inspired by trust-region ideas. The original paper (Schulman et al., 2017) calls this L^{CLIP}, but it's an objective we **maximize**, so we use J:

$$J(\theta) = \mathbb{E}_t\left[\min\left(\rho_t(\theta)\hat{A}_t,\ \text{clip}(\rho_t(\theta),\, 1-\varepsilon,\, 1+\varepsilon)\,\hat{A}_t\right)\right]$$

Where $\rho_t(\theta) = \frac{\pi_\theta(a_t|s_t)}{\pi_{\theta_{\text{old}}}(a_t|s_t)}$ is the importance-sampling ratio.

Where ε is typically 0.1–0.2. The $\min$ selects the **more conservative** estimate.

Positive advantage ($\hat{A}_t > 0$)

$$J(\theta) = \mathbb{E}_t\left[\min\left(\rho_t(\theta)\hat{A}_t,\ \text{clip}(\rho_t(\theta),\, 1-\varepsilon,\, 1+\varepsilon)\,\hat{A}_t\right)\right]$$

The action is better than average at that state — we want to **increase** its likelihood. Three sub-cases:

1. $\rho_t < 1 - \varepsilon$: Action is **less likely** under new policy.
   Objective: $\rho_t \hat{A}_t$ — **normal gradient**, push probability up
2. $1 - \varepsilon \leq \rho_t \leq 1 + \varepsilon$: Action is **roughly equally likely**.
   Objective: $\rho_t \hat{A}_t$ — **normal gradient**, push probability up
3. $\rho_t > 1 + \varepsilon$: Action is **already more likely** under new policy.
   Objective: $(1+\varepsilon)\hat{A}_t$ — gradient is **zero**, no update needed (**CLIPPED**)

Note: on the first gradient step, $\rho_t = 1$ (policy hasn't changed yet), so the ratio starts near 1 and cases 1/3 only arise on subsequent steps.

$\rho_t(\theta) = \pi_\theta / \pi_{\theta_{\text{old}}}$ (policy ratio)

Negative advantage ($\hat{A}_t < 0$)

$$J(\theta) = \mathbb{E}_t\left[\min\left(\rho_t(\theta)\hat{A}_t,\ \text{clip}(\rho_t(\theta),\, 1-\varepsilon,\, 1+\varepsilon)\,\hat{A}_t\right)\right]$$

The action was worse than average at that state — we want to **decrease** its likelihood. Three sub-cases:

1. $\rho_t < 1 - \varepsilon$: Action is **already less likely** under new policy.
   Objective: $(1-\varepsilon)\hat{A}_t$ — gradient is **zero**, no update needed (**CLIPPED**)
2. $1 - \varepsilon \leq \rho_t \leq 1 + \varepsilon$: Action is **roughly equally likely**.
   Objective: $\rho_t \hat{A}_t$ — **normal gradient**, push probability down
3. $\rho_t > 1 + \varepsilon$: Action is **more likely** under new policy.
   Objective: $\rho_t \hat{A}_t$ — **normal gradient**, push probability down

Note: on the first gradient step, $\rho_t = 1$ (policy hasn't changed yet), so the ratio starts near 1 and cases 1/3 only arise on subsequent steps.

$\rho_t(\theta) = \pi_\theta / \pi_{\theta_{\text{old}}}$ (policy ratio)

Zero advantage ($\hat{A}_t = 0$)

The action was exactly as good as expected. The loss is zero — **no update**.

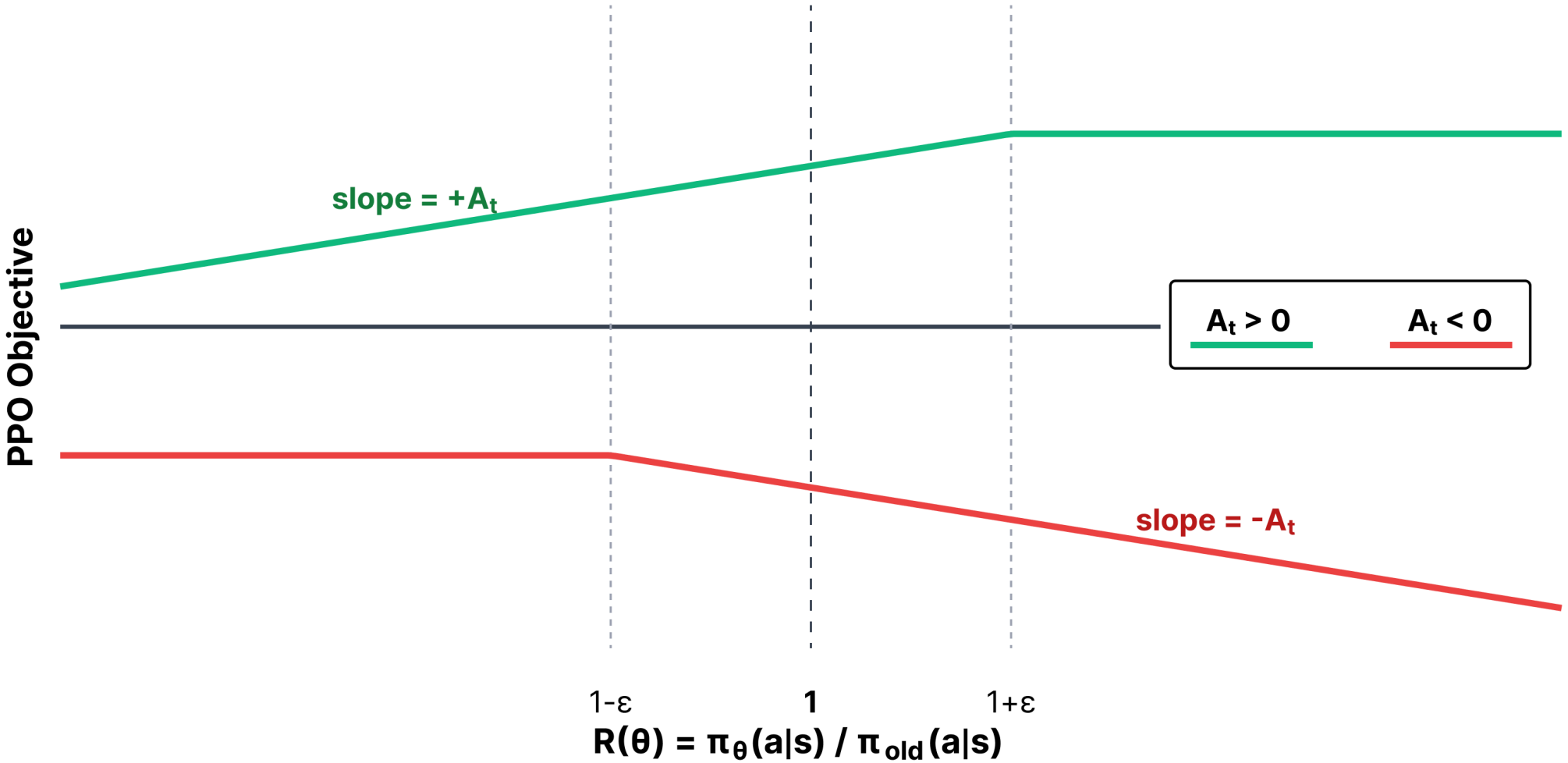
Clipping summary

In all cases, clipping stops the gradient when the policy has **already moved enough** in the right direction:

Advantage	Within trust region	Outside trust region
$\hat{A}_t > 0$	Normal gradient (reinforce)	Zero gradient if $\rho_t > 1 + \varepsilon$
$\hat{A}_t < 0$	Normal gradient (suppress)	Zero gradient if $\rho_t < 1 - \varepsilon$
$\hat{A}_t = 0$	No update	No update

The clipping is **one-sided per advantage sign**: it caps movement when the policy has already moved enough in the beneficial direction, but never blocks correction in the detrimental direction.

PPO clipping visualized



Visualization of the PPO objective for positive and negative advantage. The trust region is where the ratio is within .

The value function (critic)

PPO trains a **value function** $V_\phi(s)$ alongside the policy — the expected future return from state s_t:

$$V_\phi(s_t) = \mathbb{E}\left[\sum_{k=0}^{T-t} \gamma^k r_{t+k} \,\middle|\, s_t\right]$$

- Separate parameters ϕ (often initialized from the reward model or SFT model)
- Predicts post-KL shaped future return at each token position
- Trained via MSE against post-KL returns: $\mathcal{L}_V = \frac{1}{2}(V_\phi(s_t) - G_t)^2$

The value function serves as a learned baseline for advantage estimation.

How PPO-RLHF gets token-level credit

PPO-style RLHF often starts with one scalar reward for the whole completion. How does that become a per-token training signal?

1. **KL penalty shapes intermediate tokens**: at each token t, subtract $\beta \cdot \mathrm{KL}_t$ from the reward → per-token reward signal
2. **Final token gets the RM score**: $r_T = R(\tau) - \beta\,\mathrm{KL}_T$, all other tokens get $r_t = -\beta\,\mathrm{KL}_t$
3. **GAE propagates credit backward**: the value function + TD residuals assign per-token advantages from these shaped rewards

This is how PPO-RLHF turns a sequence-level reward into token-level training.

Advantage estimation

The simplest advantage: $\hat{A}_t = G_t - V_\phi(s_t)$

Why advantages help:

- **Centering** reduces variance — we're asking "how much better is this action than average?" rather than "how good was the total return?"
- **Credit assignment** — with per-token values, each token gets its own advantage signal

This Monte Carlo estimate is simple and unbiased, but it can be high variance. Temporal Difference (TD) methods and Generalized Advantage Estimation (GAE) trade some bias for lower variance.

The TD residual

The temporal difference (TD) residual measures how much the actual reward exceeded the value prediction:

$$\delta_t^V = r_t + \gamma V_\phi(s_{t+1}) - V_\phi(s_t)$$

This is the **1-step advantage estimate**: low variance (uses learned V_ϕ) but potentially high bias (if V_ϕ is inaccurate).

Schulman et al., 2015

GAE: K-step advantage

We can extend to K steps:

$$\hat{A}_t^{(1)} = \delta_t^V \qquad \text{(1-step: low variance, high bias)}$$

$$\hat{A}_t^{(2)} = \delta_t^V + \gamma\delta_{t+1}^V \qquad \text{(2-step)}$$

$$\hat{A}_t^{(k)} = \sum_{l=0}^{k-1} \gamma^l \delta_{t+l}^V \qquad \text{(k-step: more variance, less bias)}$$

As $k \to \infty$, we recover the full Monte Carlo advantage $G_t - V_\phi(s_t)$ (no bias, highest variance).

GAE: Exponential weighting

GAE uses an exponentially-weighted average across all K-step estimates:

$$\hat{A}_t^{\text{GAE}} = \sum_{l=0}^{\infty} (\gamma\lambda)^l \delta_{t+l}^V$$

Where $\lambda \in [0, 1]$ controls the bias-variance tradeoff.

GAE: Bias-variance tradeoff

$$\hat{A}_t^{\text{GAE}} = \sum_{l=0}^{\infty} (\gamma\lambda)^l \delta_{t+l}^V$$

λ	**Behavior**	**Variance**	**Bias**
0	Pure TD (1-step)	Lowest	Highest
0.95	Typical default for LLMs	Balanced	Balanced
1	Monte Carlo advantage	Highest	None

The γ here is typically 1.0 for language models (no discounting). These rankings assume an accurate V_ϕ — in practice, a poorly trained critic can make even low-λ estimates unreliable.

PPO-RLHF: Full policy objective

A schematic high-level view of PPO-RLHF combines the clipped PPO objective with a KL regularizer:

$$J(\theta) \approx L^{CLIP}(\theta) - \beta\, D_{\mathrm{KL}}[\pi_\theta || \pi_{\mathrm{ref}}]$$

Two layers of regularization:

1. **Clipping** — limits how far the policy moves per batch (trust region)
2. **KL penalty** — limits total drift from the reference policy across training

These serve different purposes and are not redundant.

PPO training loop (conceptual)

The PPO training loop:

1. **Generate**: sample prompts, generate completions with current policy π_θ
2. **Score**: compute rewards with reward model + KL penalty from π_{ref}
3. **Estimate advantages**: compute GAE using learned value function V_ϕ
4. **Update** (K epochs): clipped policy gradient + value function loss on the same batch
5. **Repeat**: new batch, sync $\pi_{\theta_{\text{old}}} \leftarrow \pi_\theta$

Typical: K = 2–4 gradient steps per batch before re-generating.

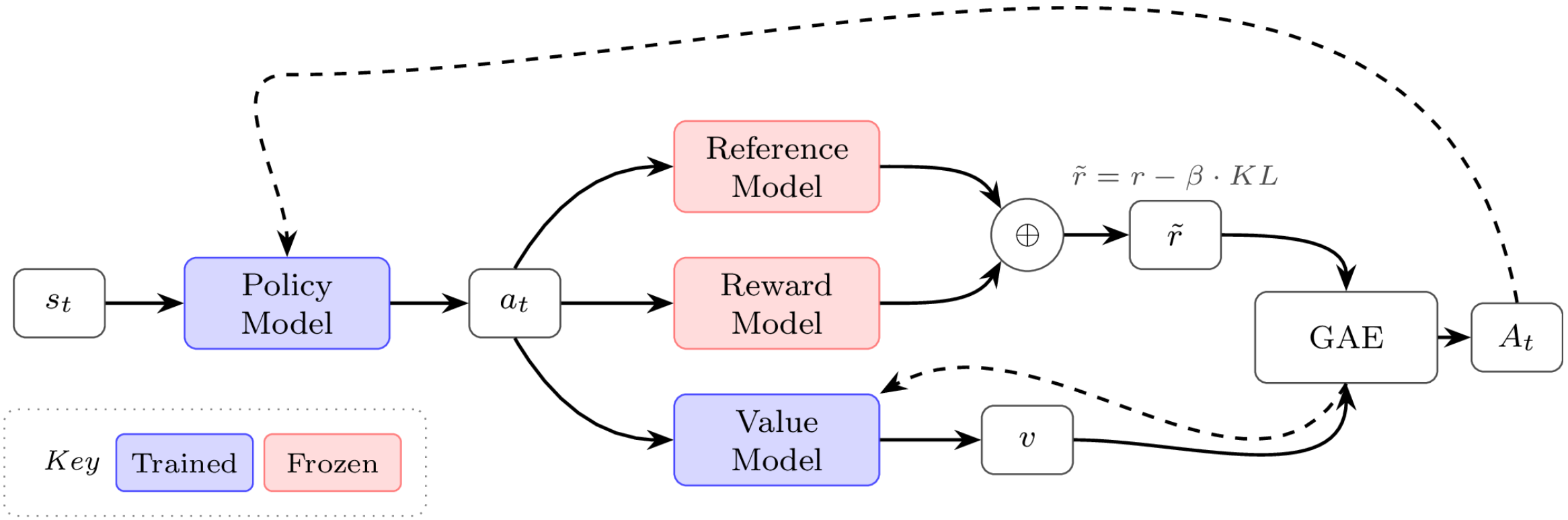
PPO: Four models in memory

PPO requires **four** models:

Model	Purpose	Updates?
Policy π_θ	Generates completions	Yes
Value function V_ϕ	Estimates per-token expected return	Yes
Reference policy π_{ref}	KL penalty anchor	Frozen
Reward model r_ψ	Scores completions	Frozen

This is memory-intensive — a key motivation for simpler alternatives like GRPO.

PPO architecture



PPO architecture for language models. Adds a trained value model and GAE for per-token advantage estimation.

GRPO & modern variants

GRPO: PPO for the reasoning era

Group Relative Policy Optimization (GRPO) was introduced in DeepSeekMath (Shao et al., 2024) for math reasoning and has since become the go-to algorithm for RL on language models. It keeps PPO's clipped objective but drops the value function entirely.

Core idea: generate G completions per prompt, use the group's reward statistics as the baseline — no learned critic needed.

- Simpler to implement and debug than PPO
- Saves memory (one fewer model copy)
- Natural fit for RLVR where you have a clear verification signal and higher variance rewards (e.g. 0 or 1 for correctness)
- Popularized with DeepSeek R1 and everything that followed

Shao et al., 2024

GRPO advantage

For a group of G completions with rewards $R_1, \ldots, R_G$:

$$\hat{A}_i = \frac{R_i - \text{mean}(R_1, \ldots, R_G)}{\text{std}(R_1, \ldots, R_G)}$$

Z-score normalization: positive advantage for above-average completions, negative for below.

Each token in completion i gets the **same** advantage (sequence-level, not per-token).

GRPO objective

Clipped ratio (like PPO) + group-normalized advantages + KL penalty directly in loss:

$$J(\theta) = \frac{1}{G}\sum_{i=1}^{G}\frac{1}{|a_i|}\sum_{t=1}^{|a_i|}\left(\min\left(\rho_{i,t}\hat{A}_i,\ \text{clip}(\rho_{i,t}, 1-\varepsilon, 1+\varepsilon)\hat{A}_i\right) - \beta\, D_{\text{KL},i,t}\right)$$

$$\hat{A}_i = \frac{R_i - \text{mean}(R_1, \ldots, R_G)}{\text{std}(R_1, \ldots, R_G)}$$

Where the clipping applies **per-token**: $\rho_{i,t} = \frac{\pi_\theta(a_{i,t}|s_t)}{\pi_{\theta_{\text{old}}}(a_{i,t}|s_t)}$, but $\hat{A}_i$ is shared across all tokens in the completion (sequence-level advantage, per-token ratio).

GRPO vs PPO

	PPO	GRPO
Value function	Learned V_ϕ	None
Advantage	Per-token via GAE	Sequence-level, group z-score
KL penalty	In reward (before advantages)	In loss (default, but optional)
Models in memory	4 (policy, value, ref, RM)	3 (policy, ref, RM) or 2 without KL
Complexity	Higher	Lower
Popular for	General RLHF	Reasoning / RLVR (DeepSeek R1)

GRPO is PPO minus the value function, with a statistical baseline instead.

RLOO vs GRPO

Both use multiple completions per prompt. The key difference is in the PPO-style clipping for GRPO:

	RLOO	GRPO
Baseline	Leave-one-out mean	Group mean (z-scored)
Update style	REINFORCE (no clipping)	PPO-style clipped ratio
KL penalty	Optional (in reward)	In loss (default, but optional)
Advantage	$R_k - \frac{1}{K-1}\sum_{j \neq k} R_j$	$\frac{R_i - \text{mean}}{\text{std}}$

Same principle (compare to peers), different mechanics. Without std normalization, the GRPO-style advantage estimate becomes equivalent to RLOO up to a scaling constant.

GSPO: Sequence-level ratios

Problem: aggregating per-token importance ratios across long sequences is numerically unstable. A single token with a large ratio can dominate the update.

GSPO uses a geometric mean — a single, length-normalized importance weight per response:

$$\rho_i(\theta) = \exp\left(\frac{1}{|a_i|}\sum_{t=1}^{|a_i|}\log\frac{\pi_\theta(a_{i,t} \mid s)}{\pi_{\theta_{\text{old}}}(a_{i,t} \mid s)}\right)$$

The geometric mean stays in a reasonable numerical range for any sequence length — similar gradient signal, better numerics. The full objective mirrors GRPO but with the sequence-level ratio:

$$J_{\text{GSPO}}(\theta) = \frac{1}{G}\sum_{i=1}^{G}\min\Big(\rho_i(\theta)\hat{A}_i,\ \text{clip}(\rho_i(\theta), 1-\varepsilon, 1+\varepsilon)\hat{A}_i\Big)$$

The clipping range ε now operates on a per-token average scale, making it comparable across different completion lengths.

Zheng et al., 2025

CISPO: Clipped importance sampling

CISPO clips the **importance weights themselves** rather than the objective, using a stop-gradient:

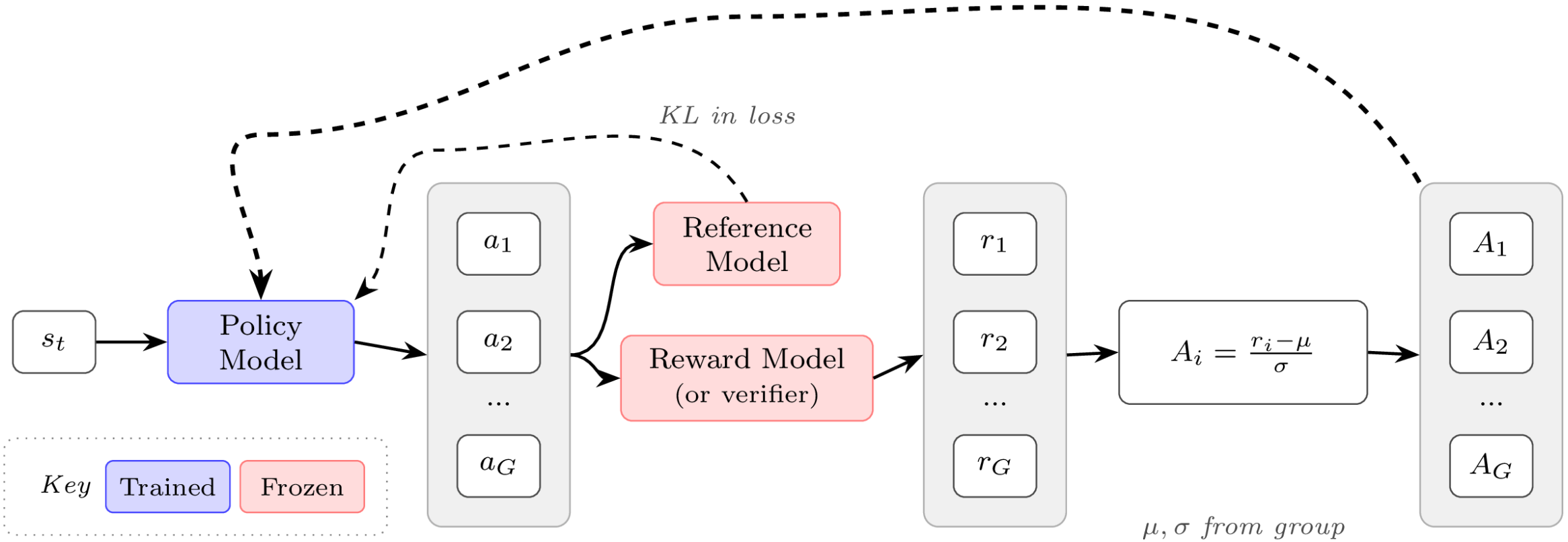
$$J_{\text{CISPO}}(\theta) = \sum_{i,t} \text{sg}(\hat{\rho}_{i,t})\, A_{i,t} \log \pi_\theta(a_{i,t} \mid s_t)$$

$$\hat{\rho}_{i,t} = \text{clip}\left(\frac{\pi_\theta(a_{i,t} \mid s_t)}{\pi_{\theta_{\text{old}}}(a_{i,t} \mid s_t)},\ 1-\varepsilon^-,\ 1+\varepsilon^+\right)$$

Key difference from PPO: every token still receives a gradient signal — the weight just bounds how much it's amplified. Asymmetric bounds ($\varepsilon^+ > \varepsilon^-$) allow more aggressive reward-increasing updates, encouraging exploration.

MiniMax Team, 2025

GRPO architecture



GRPO architecture for language models. Multiple completions per prompt, group-normalized advantages, no value model needed.

Conclusion

Lecture summary

In most RLHF setups, data quality and reward signal quality dominate; algorithm choice mostly determines stability, efficiency, and engineering burden. All methods optimize the **same** policy gradient objective:

$$\nabla_\theta J(\theta) = \mathbb{E}_{\tau \sim p_\theta}\left[\sum_t \Psi_t \nabla_\theta \log \pi_\theta(a_t \mid s_t)\right]$$

They differ in:

1. **What Ψ_t is**: total return, advantage, group-normalized advantage
2. **How updates are bounded**: no clipping (REINFORCE), objective clipping (PPO/GRPO), weight clipping (CISPO)
3. **Memory and compute**: 2–4 models, value function training or not

RL cheatsheet

A one-page reference of all core RL loss functions is available at:

rlhfbook.com/rl-cheatsheet

Next lecture: RL implementation

Lecture 4 turns these algorithms into working code:

- **Core loss implementations**: Setting up RL algorithms with language models
- **Loss aggregation**: per-sequence vs per-token normalization and why it matters
- **Infra setup**: what to store (log-probs, values, ref log-probs) and what goes wrong when you don't
- **Trade-offs**: monitoring training, identifying reward hacking, fixing common failures

Thank you

Questions / discussion

Contact: nathan@natolambert.com

Newsletter: interconnects.ai

rlhfbook.com



References (1/2)

Ahmadian, A., Cremer, C., Gall\"e, M., Fadaee, M., Kreutzer, J., et al. "*Back to Basics: Revisiting REINFORCE-Style Optimization for Learning from Human Feedback in LLMs*." *arXiv preprint arXiv:2402.14740*, 2024.

Christiano, P., Leike, J., Brown, T., Martic, M., Legg, S., et al. "*Deep Reinforcement Learning from Human Preferences*." *Advances in Neural Information Processing Systems*, 2017.

Lambert, N., Morrison, J., Pyatkin, V., Huang, S., Ivison, H., et al. "*Tülu 3: Pushing Frontiers in Open Language Model Post-Training*." *arXiv preprint arXiv:2411.15124*, 2024.

MiniMax Team. "*MiniMax-01: Scaling Foundation Models with Lightning Attention*." *arXiv preprint arXiv:2501.08313*, 2025.

OpenAI. "*Introducing OpenAI o1-preview*." *OpenAI Blog*, 2024. [link]

Ouyang, L., Wu, J., Jiang, X., Almeida, D., Wainwright, C., et al. "*Training language models to follow instructions with human feedback*." *Advances in Neural Information Processing Systems*, 2022.

Schulman, J., Moritz, P., Levine, S., Jordan, M., and Abbeel, P. "*High-Dimensional Continuous Control Using Generalized Advantage Estimation*." *arXiv preprint arXiv:1506.02438*, 2015.

Schulman, J., Levine, S., Abbeel, P., Jordan, M., and Moritz, P. "*Trust Region Policy Optimization*." *International Conference on Machine Learning*, 2015.

References (2/2)

Schulman, J., Wolski, F., Dhariwal, P., Radford, A., and Klimov, O. "*Proximal Policy Optimization Algorithms*." *arXiv preprint arXiv:1707.06347*, 2017.

Shao, Z., Wang, P., Zhu, Q., Xu, R., Song, J., et al. "*DeepSeekMath: Pushing the Limits of Mathematical Reasoning in Open Language Models*." *arXiv preprint arXiv:2402.03300*, 2024.

Wang, Z., Dong, Y., Delalleau, O., Zeng, J., Shen, G., et al. "*HelpSteer2-Preference: Complementing Ratings with Preferences*." *arXiv preprint arXiv:2410.01257*, 2024.

Williams, R. "*Simple Statistical Gradient-Following Algorithms for Connectionist Reinforcement Learning*." *Machine Learning*, 1992.

Zheng, Z., Guo, D., and others. "*Group Sequence Policy Optimization*." *arXiv preprint*, 2025.